MODUL PRAKTIKUM 7
PENGELOLAAN STATE DAN VALIDASI INPUT

7.1. Topik Bahasan

Modul ini dilaksanakan pada pertemuan ke-7 sesuai Rencana Pembelajaran Semester mata kuliah Pemrograman IV dan membebani CPMK-02, dengan cakupan topik sebagai berikut:

- Perubahan *state* melalui `setState` pada `StatefulWidget`.
- Pemisahan data dan tampilan serta pengangkatan state ke *widget* induk.
- Penyusunan formulir pemesanan pengiriman menggunakan `Form` dan `TextFormField`.
- Penerapan validator pada masukan wajib, format angka, dan tanggal.
- Pengiriman hasil formulir kembali ke halaman pemanggil.

7.2. Capaian Pembelajaran

Mahasiswa mampu mengelola state aplikasi dan memastikan *input* pengguna yang valid (CPMK-02).

7.3. Indikator Ketercapaian Pembelajaran

Indikator penilaian pertemuan ke-7 menurut Rencana Pembelajaran Semester adalah:

- Ketepatan pengelolaan state pada perubahan data antarmuka
- Ketepatan penerapan validasi input

Indikator tersebut dinyatakan tercapai apabila bukti berikut terlihat pada hasil praktikum:

- Perubahan data langsung tampil pada antarmuka tanpa memuat ulang aplikasi.
- Input yang tidak valid ditolak disertai pesan *error* yang jelas pada tiap *field*.
- Data yang valid tersimpan pada objek model dan muncul pada daftar pengiriman.
- Pengelola teks (`TextEditingController`) dilepas dengan benar pada metode `dispose`.

7.4. Teori Pendukung

Konsep State

State adalah data yang dapat berubah selama aplikasi berjalan dan memengaruhi tampilan. Pada Flutter, perubahan tampilan tidak dilakukan dengan mengubah widget secara langsung, melainkan dengan mengubah data lalu meminta *framework* membangun ulang widget. Permintaan tersebut disampaikan melalui pemanggilan

`setState`. Kode yang mengubah data harus diletakkan di dalam `setState`, bukan di luarnya, agar Flutter mengetahui bahwa pembangunan ulang diperlukan.

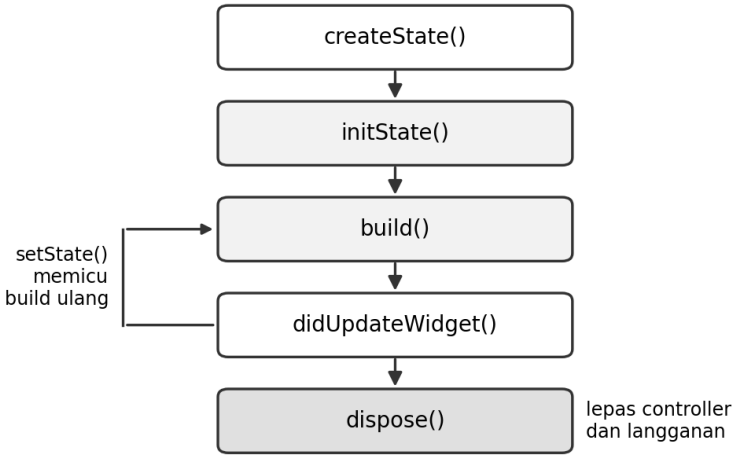
Metode `build` dapat dipanggil berkali-kali dalam waktu singkat. Karena itu metode ini harus bebas dari efek samping: tidak boleh melakukan pemanggilan jaringan, membaca basis data, atau mengubah data. Seluruh proses semacam itu diletakkan pada `initState`, pada penangan peristiwa, atau pada lapisan terpisah.

Siklus Hidup StatefulWidget

- `createState()` membentuk objek `State` ketika widget pertama kali dimasukkan ke *widget tree*.
- `initState()` dijalankan satu kali dan merupakan tempat yang tepat untuk menyiapkan pengendali serta memuat data awal.
- `build()` dijalankan setiap kali widget perlu digambar ulang.
- `didUpdateWidget()` dijalankan ketika widget induk membangun ulang widget ini dengan data baru.
- `dispose()` dijalankan ketika widget dikeluarkan dari widget tree dan merupakan tempat wajib untuk melepas `TextEditingController`, `FocusNode`, serta langganan aliran data.

Tabel 8. Metode siklus hidup StatefulWidget dan penggunaannya

Metode	Dijalankan ketika	Isi yang tepat
`createState()`	Widget pertama kali masuk widget tree	Membentuk objek State
`initState()`	Sekali, sebelum build pertama	Menyiapkan controller dan memuat data awal
`build()`	Setiap kali widget perlu digambar ulang	Menyusun widget, tanpa efek samping
`didUpdateWidget()`	Induk membangun ulang dengan data baru	Menyesuaikan state terhadap data baru
`dispose()`	Widget dikeluarkan dari widget tree	Melepas controller, focus node, dan langganan



Gambar 8. Siklus hidup StatefulWidget dan titik pemanggilan setState

Pengangkatan State

Ketika dua widget bersaudara membutuhkan data yang sama, data tersebut dipindahkan ke widget induk terdekat yang menaungi keduanya. Teknik ini disebut pengangkatan state (lifting state up). Widget anak kemudian menerima data melalui parameter dan menyampaikan peristiwa melalui fungsi panggil balik (callback). Pola ini menjaga agar sumber kebenaran data hanya satu.

Untuk aplikasi berskala lebih besar, pola ini biasanya dilengkapi *library* pengelolaan state seperti Provider, Riverpod, atau BLoC. Pada praktikum ini pembahasan dibatasi pada `setState` dan pengangkatan state agar dasar mekanismenya benar-benar dipahami sebelum memakai library tambahan.

Formulir dan Validasi

Widget `Form` bertindak sebagai wadah yang mengoordinasikan sekumpulan field. Setiap `TextFormField` dapat memiliki fungsi `validator` yang mengembalikan `null` bila input sah, atau mengembalikan teks pesan error bila tidak sah. Pemanggilan `_formKey.currentState!.validate()` menjalankan seluruh validator sekaligus dan mengembalikan nilai `true` hanya bila semuanya lolos.

Validasi pada sisi antarmuka merupakan lapisan pertama, bukan satu-satunya. Aturan bisnis yang penting, misalnya jumlah barang tidak melebihi stok gudang, tetap harus diperiksa pada lapisan model atau layanan agar tidak dapat ditembus melalui jalur lain.

Properti `keyboardType` dan `inputFormatters` membantu mengarahkan pengguna agar mengetikkan data dengan format yang benar sejak awal, sehingga jumlah kesalahan input berkurang.

7.5. Alat Pendukung

Perangkat keras, perangkat lunak, dan bahan yang diperlukan pada modul ini adalah:

- Flutter SDK dan Dart SDK.
- Visual Studio Code beserta ekstensi Flutter dan Dart.
- Google Chrome atau Microsoft Edge.
- Flutter DevTools.
- Repositori GitHub pribadi praktikan (satu repositori untuk seluruh modul praktikum).

7.6. Langkah-langkah Praktikum

A. Penyiapan

1. Buka proyek `silog_app` melalui menu File > Open Folder pada Visual Studio Code, lalu buka terminal terintegrasi melalui menu Terminal > New Terminal. Buat cabang kerja modul ini pada terminal dengan perintah `git checkout -b praktikum/modul-07`, kemudian pastikan cabang aktif melalui `git branch`.
2. Pastikan hasil Modul 6 masih berjalan dengan benar melalui `flutter run -d chrome`, lalu hentikan aplikasi dengan menekan `q` pada terminal sebelum melanjutkan.

B. Mengubah Layar Daftar Menjadi StatefulWidget

1. Buka berkas `lib/tampilan/daftar_pengiriman_page.dart` melalui panel Explorer, ganti seluruh isinya dengan kode berikut agar `DaftarPengirimanPage` menjadi `StatefulWidget` dan daftar dapat difilter serta ditambah, lalu simpan melalui `Ctrl+S`.

Kode Program 7.1 Layar daftar dengan state dan filter

```
import 'package:flutter/material.dart';
import '../data/data_contoh.dart';
import '../model/pengiriman.dart';
import '../widget/kartu_pengiriman.dart';
import 'detail_pengiriman_page.dart';
import 'form_pengiriman_page.dart';

class DaftarPengirimanPage extends StatefulWidget {
  const DaftarPengirimanPage({super.key});

  @override
  State<DaftarPengirimanPage> createState() => _DaftarPengirimanPageState();
}

class _DaftarPengirimanPageState extends State<DaftarPengirimanPage> {
  late List<Pengiriman> _daftar;
  String _filterStatus = 'Semua';

  @override
  void initState() {
    super.initState();
    _daftar = List<Pengiriman>.from(dataPengiriman);
  }
```

```
List<Pengiriman> get _hasilSaring => _filterStatus == 'Semua'
    ? _daftar
    : _daftar.where((p) => p.status == _filterStatus).toList();

Future<void> _bukaFormulir() async {
  final hasil = await Navigator.push<Pengiriman>(
    context,
    MaterialPageRoute(builder: (_) => const FormPengirimanPage()),
  );

  if (hasil != null) {
    setState(() => _daftar.insert(0, hasil));
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Pengiriman ${hasil.resi} berhasil ditambahkan')),
    );
  }
}

@override
Widget build(BuildContext context) {
  const opsi = ['Semua', 'Menunggu Pickup', 'Dalam Perjalanan', 'Diterima'];

  return Scaffold(
    appBar: AppBar(title: const Text('Daftar Pengiriman')),
    body: Column(
      children: [
        SingleChildScrollView(
          scrollDirection: Axis.horizontal,
          padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
          child: Row(
            children: opsi.map((o) {
              return Padding(
                padding: const EdgeInsets.only(right: 8),
                child: ChoiceChip(
                  label: Text(o),
                  selected: _filterStatus == o,
                  onSelected: (_) => setState(() => _filterStatus = o),
                ),
              );
            }).toList(),
          ),
        ),
        Text('Menampilkan ${_hasilSaring.length} dari ${_daftar.length} data'),
        Expanded(
          child: _hasilSaring.isEmpty
              ? const Center(child: Text('Tidak ada data pengiriman'))
              : ListView.builder(
                  itemCount: _hasilSaring.length,
                  itemBuilder: (context, i) {
                    final item = _hasilSaring[i];
                    return KartuPengiriman(
                      pengiriman: item,
                      onTap: () => Navigator.push(
                        context,
                        MaterialPageRoute(
                          builder: (_) =>
                              DetailPengirimanPage(pengiriman: item),
                        ),
                      ),
                    );
                  },
                ),
        ),
      ],
```

```
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _bukaFormulir,
        icon: const Icon(Icons.add),
        label: const Text('Pengiriman Baru'),
      ),
    );
  }
}
```

C. Formulir Pemesanan Pengiriman

1. Buat berkas `lib/tampilan/form_pengiriman_page.dart` melalui panel Explorer: klik kanan direktori `lib/tampilan`, pilih New File, lalu ketik nama berkas `form_pengiriman_page.dart`. Ketik kode berikut, kemudian simpan melalui `Ctrl+S`.

Kode Program 7.2 Formulir dengan validasi input

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import '../data/data_contoh.dart';
import '../model/barang.dart';
import '../model/pengiriman.dart';

class FormPengirimanPage extends StatefulWidget {
  const FormPengirimanPage({super.key});

  @override
  State<FormPengirimanPage> createState() => _FormPengirimanPageState();
}

class _FormPengirimanPageState extends State<FormPengirimanPage> {
  final _formKey = GlobalKey<FormState>();
  final _resiC = TextEditingController();
  final _namaBarangC = TextEditingController();
  final _hargaC = TextEditingController();
  final _jumlahC = TextEditingController();

  String _kotaTujuan = 'Surabaya';
  DateTime _tanggalKirim = DateTime.now();

  @override
  void dispose() {
    _resiC.dispose();
    _namaBarangC.dispose();
    _hargaC.dispose();
    _jumlahC.dispose();
    super.dispose();
  }

  String? _wajib(String? v, String label) {
    if (v == null || v.trim().isEmpty) return '$label wajib diisi';
    return null;
  }

  String? _validasiResi(String? v) {
    final dasar = _wajib(v, 'Nomor resi');
    if (dasar != null) return dasar;
    final pola = RegExp(r'^SLG-\d{4}-\d{4}$');
    if (!pola.hasMatch(v!.trim())) {
      return 'Format resi harus SLG-TAHUN-NOMOR, contoh SLG-2026-0010';
    }
    return null;
```

```
  }

  String? _validasiAngka(String? v, String label, {double min = 1}) {
    final dasar = _wajib(v, label);
    if (dasar != null) return dasar;
    final angka = double.tryParse(v!.trim());
    if (angka == null) return '$label harus berupa angka';
    if (angka < min) return '$label minimal $min';
    return null;
  }

  String? _validasiTanggal() {
    final batas = DateTime.now().subtract(const Duration(days: 1));
    if (_tanggalKirim.isBefore(batas)) {
      return 'Tanggal kirim tidak boleh sebelum hari ini';
    }
    return null;
  }

  Future<void> _pilihTanggal() async {
    final hasil = await showDatePicker(
      context: context,
      initialDate: _tanggalKirim,
      firstDate: DateTime.now(),
      lastDate: DateTime.now().add(const Duration(days: 90)),
    );
    if (hasil != null) setState(() => _tanggalKirim = hasil);
  }

  void _simpan() {
    final pesanTanggal = _validasiTanggal();
    if (!_formKey.currentState!.validate() || pesanTanggal != null) {
      if (pesanTanggal != null) {
        ScaffoldMessenger.of(context)
            .showSnackBar(SnackBar(content: Text(pesanTanggal)));
      }
      return;
    }

    final pengiriman = Pengiriman(
      resi: _resiC.text.trim(),
      asal: gudangBandung,
      kotaTujuan: _kotaTujuan,
      barang: Barang(
        kode: 'B-${DateTime.now().millisecondsSinceEpoch % 10000}',
        nama: _namaBarangC.text.trim(),
        hargaSatuan: double.parse(_hargaC.text.trim()),
      ),
      jumlah: int.parse(_jumlahC.text.trim()),
      tanggalKirim: _tanggalKirim,
      status: 'Menunggu Pickup',
    );

    Navigator.pop(context, pengiriman);
  }

  @override
  Widget build(BuildContext context) {
    const kota = ['Surabaya', 'Makassar', 'Medan', 'Denpasar', 'Jayapura'];

    return Scaffold(
      appBar: AppBar(title: const Text('Pengiriman Baru')),
      body: Form(
        key: _formKey,
        autovalidateMode: AutovalidateMode.onUserInteraction,
        child: ListView(
```

```
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _resiC,
              decoration: const InputDecoration(
                labelText: 'Nomor Resi',
                hintText: 'SLG-2026-0010',
                border: OutlineInputBorder(),
              ),
              textCapitalization: TextCapitalization.characters,
              validator: _validasiResi,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _namaBarangC,
              decoration: const InputDecoration(
                labelText: 'Nama Barang',
                border: OutlineInputBorder(),
              ),
              validator: (v) => _wajib(v, 'Nama barang'),
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _hargaC,
              decoration: const InputDecoration(
                labelText: 'Harga Satuan (Rp)',
                border: OutlineInputBorder(),
              ),
              keyboardType: TextInputType.number,
              inputFormatters: [FilteringTextInputFormatter.digitsOnly],
              validator: (v) => _validasiAngka(v, 'Harga satuan', min: 1000),
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _jumlahC,
              decoration: const InputDecoration(
                labelText: 'Jumlah',
                border: OutlineInputBorder(),
              ),
              keyboardType: TextInputType.number,
              inputFormatters: [FilteringTextInputFormatter.digitsOnly],
              validator: (v) => _validasiAngka(v, 'Jumlah'),
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<String>(
              value: _kotaTujuan,
              decoration: const InputDecoration(
                labelText: 'Kota Tujuan',
                border: OutlineInputBorder(),
              ),
              items: kota
                  .map((k) => DropdownMenuItem(value: k, child: Text(k)))
                  .toList(),
              onChanged: (v) => setState(() => _kotaTujuan = v!),
            ),
            const SizedBox(height: 12),
            OutlinedButton.icon(
              onPressed: _pilihTanggal,
              icon: const Icon(Icons.calendar_today),
              label: Text(
                'Tanggal Kirim:
${_tanggalKirim.day}/${_tanggalKirim.month}/${_tanggalKirim.year}',
              ),
            ),
            const SizedBox(height: 24),
            FilledButton(
```

```
              onPressed: _simpan,
              child: const Text('Simpan Pengiriman'),
            ),
          ],
        ),
      ),
    );
  }
}
```

D. Pengujian

1. Jalankan aplikasi melalui `flutter run -d chrome`.
2. Tekan tombol Pengiriman Baru, lalu tekan Simpan tanpa mengisi apa pun. Catat pesan error yang muncul pada tiap field.
3. Isi nomor resi dengan format keliru, misalnya `ABC-1`, dan pastikan pesan error format muncul.
4. Isi seluruh field dengan data yang sah, simpan, lalu pastikan data baru muncul di urutan teratas daftar dan pesan konfirmasi tampil.
5. Cobalah *filter* status melalui *dropdown* pada bagian atas layar daftar, lalu pastikan jumlah data yang ditampilkan berubah sesuai pilihan. Hentikan aplikasi dengan menekan `q` pada terminal setelah pengujian selesai.

E. Tugas Praktikum

1. Tambahkan field berat barang dalam kilogram beserta validasi nilai antara 0,1 dan 5000.
2. Tambahkan pemeriksaan agar nomor resi yang sudah ada pada daftar tidak dapat dipakai ulang, disertai pesan error yang sesuai.
3. Tambahkan tombol hapus pada layar detail yang menghapus data dari daftar setelah dikonfirmasi melalui `AlertDialog`.
4. Jelaskan pada jurnal mengapa `TextEditingController` wajib dilepas pada `dispose` dan apa akibatnya bila dilewatkan.

F. Luaran yang Dinilai

Bentuk penilaian modul ini adalah tugas praktikum dengan kriteria keberfungsian formulir beserta validasinya. jurnal praktikum individu diunggah ke lms ulbi paling lambat 5x24 jam setelah sesi praktikum. Agar kriteria tersebut dapat dinilai, praktikan menyerahkan luaran berikut:

- Berkas `form_pengiriman_page.dart` dan `daftar_pengiriman_page.dart`.
- Tangkapan layar pesan error yang muncul pada setiap field ketika formulir dikosongkan.
- Tangkapan layar penolakan format nomor resi yang tidak sah.
- Tangkapan layar data baru yang berhasil tersimpan dan muncul pada urutan teratas daftar.

- Tabel kasus uji validasi berisi field, input uji, aturan validasi, dan hasil yang diperoleh.

7.7. Panduan Pengumpulan Jurnal Hasil Kegiatan Praktikum

Tugas praktikum modul ini bersifat individu. Setiap praktikan menyusun dan mengunggah sendiri jurnalnya ke LMS ULBI paling lambat 5 x 24 jam setelah sesi praktikum berakhir, berupa berkas PDF bernama `NIM_Nama_Modul-XX.pdf`, disertai tautan repositori GitHub pribadi pada cabang `praktikum/modul-XX`.

Sistematika jurnal:

1. Identitas: nama, NIM, kelas, nomor dan judul modul, serta tanggal praktikum.
2. Tujuan praktikum.
3. Alat dan bahan yang digunakan beserta versinya.
4. Langkah kerja, disertai tangkapan layar tiap tahap penting.
5. Kode program beserta penjelasan singkatnya.
6. Hasil eksekusi dan analisisnya, termasuk error yang ditemui beserta penyelesaiannya.
7. Jawaban seluruh butir Tugas Praktikum.
8. Kesimpulan dan kendala.
9. Tautan repositori beserta tangkapan layar riwayat *commit*.

Tangkapan layar wajib memperlihatkan jendela *browser* atau emulator secara utuh beserta kode programnya, dan memuat nama serta NIM praktikan. Praktikan yang tidak mengumpulkan jurnal memperoleh nilai nol; keterlambatan dan ketidaksesuaian hasil lainnya dikenai pengurangan nilai sesuai kebijakan dosen pengampu.

Seluruh butir pada sub bab Luaran yang Dinilai wajib dilampirkan pada jurnal.

7.8. Hasil Capaian Pembelajaran Praktikum

Bentuk dan kriteria penilaian mengikuti Rencana Pembelajaran Semester pertemuan ke-7: Tugas praktikum: keberfungsian formulir beserta validasinya. Jurnal praktikum individu diunggah ke LMS ULBI paling lambat 5x24 jam setelah sesi praktikum.

Tugas praktikum modul ini bersifat individu dan dinilai dari jurnal praktikum masing-masing praktikan.

Bobotnya 2% dari nilai akhir mata kuliah pada komponen evaluasi Praktikum (Aktivitas Partisipatif) sebesar 15% dari nilai akhir, diukur dari kelengkapan dan mutu luaran pada sub bab Luaran yang Dinilai terhadap indikator penilaian modul ini.